Picasso CSR Coloring on WSE-3

Design Document for SW-Relay and the Current 2D Segmented LWW Kernel

Independent Study Working Document

Last updated: April 27, 2026

Abstract

This document explains the two main designs in the repository for solving the Picasso CSR graph-coloring problem on the Cerebras WSE-3: `sw-relay`, the stable baseline, and `2d_seg2`, the current high-performance 2D design. The goal is not to repeat every debugging note. The goal is to show the layout, the route types, the named components, and the design reasons in a precise and direct form.

Contents

1 Problem Statement **3**
1.1 One Coloring Round . . . 3

2 Hardware Constraints That Shape the Design **3**

3 Design A: SW-Relay Baseline **4**
3.1 Purpose . . . 4
3.2 16 by 16 Layout . . . 4
3.3 Named Components . . . 5
3.4 Route Types . . . 5
3.5 Why We Built It This Way . . . 5

4 Design B: Current 2D Segmented LWW Design **6**
4.1 Purpose . . . 6
4.2 Core Idea: Make the Loser Receive the Winner . . . 6
4.3 16 by 16 Layout Overview . . . 6
4.4 Concrete Graph Wavelet Trace . . . 9
4.5 Named Components . . . 13
4.6 Color Allocation, Queue Budget, and Bridge Roles . . . 13
4.7 Route 1: Row Segments . . . 15
4.8 Route 2: Column Movement . . . 16
4.9 Route 3: E2S Relay and Back-Channel . . . 16
4.10 Route 4: Barriers and Broadcasts . . . 16
4.11 Wavelet Format . . . 17
4.12 Why We Arrived at This Design . . . 17

5 Design Comparison **18**

6 Current Hardware Snapshot **18**

7 Next Performance and System Work **19**
7.1 On-Device Recursion 19
7.2 Route and Kernel Performance Work 19

8 Source Map **20**

1 Problem Statement

The input is a conflict graph stored in CSR form. CSR stands for Compressed Sparse Row. It uses two arrays:

- `csr_offsets[v]` and `csr_offsets[v+1]` give the range of neighbor entries for vertex `v`.
- `csr_adj[]` stores all neighbor vertex IDs in one packed array.

The coloring task is to assign a color to every vertex so that no two adjacent vertices share a color. On the WSE-3, the graph is split across many processing elements, or PEs. Each PE stores a small local CSR shard and a boundary table for graph edges that cross to other PEs.

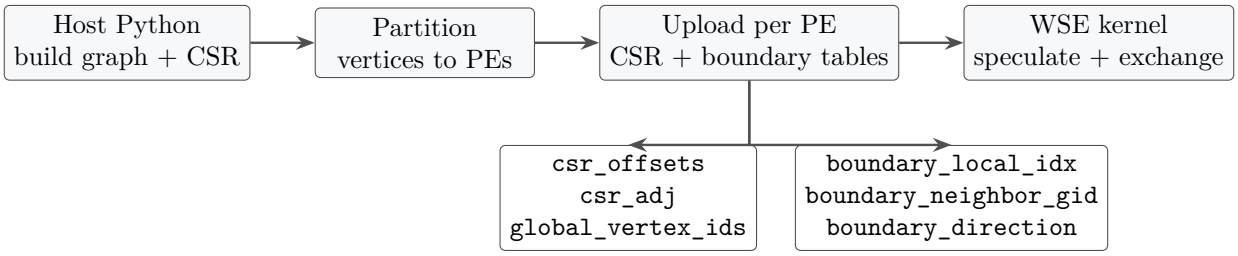


Figure 1: Common data path for both designs. The host builds the graph, partitions it, and uploads CSR and boundary data. The device kernel then runs the coloring rounds.

1.1 One Coloring Round

Both designs use the same high-level round:

1. Each PE chooses tentative colors for its local uncolored vertices.
2. Each PE sends color updates for boundary vertices to the PEs that need to see them.
3. Each PE receives remote colors and records them by sender GID.
4. Each PE detects conflicts. If a local vertex loses, it becomes uncolored again and tries in the next round.
5. A barrier checks whether any PE still has uncolored vertices.

The two designs differ almost entirely in step 2: how boundary color messages move across the WSE fabric.

2 Hardware Constraints That Shape the Design

The WSE-3 gives us many PEs, but each PE has tight local limits. These limits explain most of the design choices.

Constraint	Effect on the design
Six user input queues and six user output queues per PE	Every distinct receive or send stream consumes queue budget. Designs that need many independent streams per PE do not scale.

Constraint	Effect on the design
Finite fabric colors	A color is a route label. We cannot spend one color per PE on a large grid, so the current design reuses colors by segment.
One source per color on a route	Two PEs cannot inject unrelated wavelets onto the same live color path. Bridge PEs solve this by terminating a color and re-emitting on another.
Small per-PE SRAM	Each PE stores CSR arrays, boundary tables, colors, and runtime state. The layout must keep per-PE arrays bounded.
No shared memory	All cross-PE information must travel as wavelets on the fabric.

3 Design A: SW-Relay Baseline

3.1 Purpose

`sw-relay` is the stable baseline. It is simple and robust. It supports 1D and 2D grids because it does not rely on a complex global fabric route. When a wavelet needs to travel multiple hops, intermediate PEs receive it, inspect it, and forward it in software.

- Source files: `csl/layout.csl`, `csl/pe_program.csl`.
- Runner option: `-routing sw-relay`.
- Partitioning: hash mode in the current runner unless overridden.
- Main strength: correctness and flexibility.
- Main weakness: every hop wakes a PE task, so long routes are slow.

3.2 16 by 16 Layout

The 16 by 16 view below shows the checkerboard route pattern. Neighboring PEs use opposite parity so that a sender and receiver do not fight over the same color. The drawn diagonal path is an example of a multi-hop software relay: each hop is received by a CPU task, buffered, and re-sent.

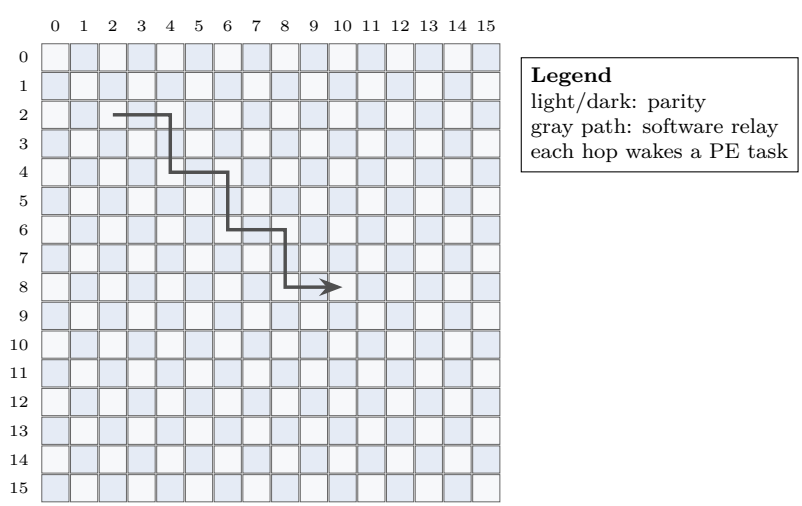
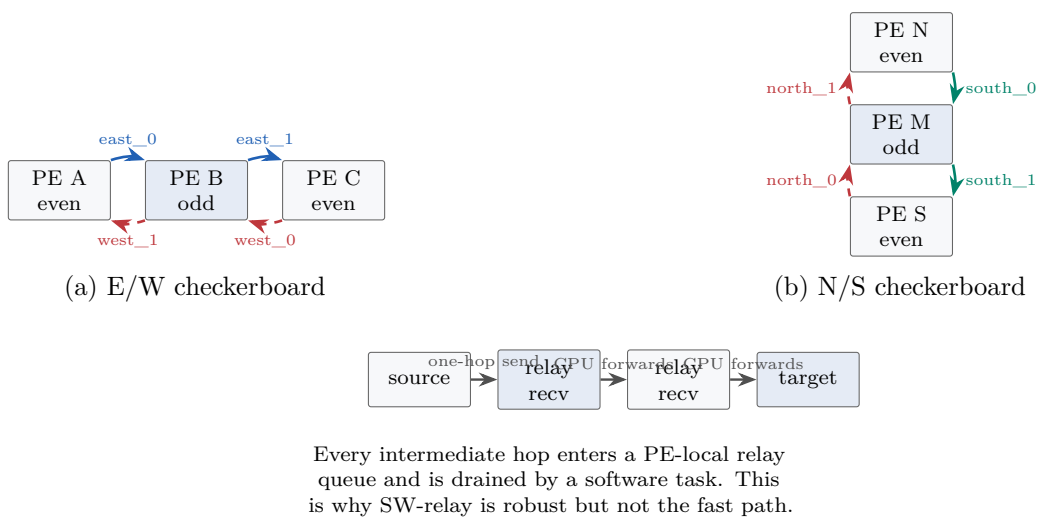


Figure 2: SW-relay on a 16 by 16 grid. Multi-hop delivery is a chain of software-forwarded one-hop moves.

3.3 Named Components

Component	Role
`csr_offsets`, `csr_adj`	Local CSR shard for vertices owned by the PE.
`boundary_*` arrays	Boundary entries that say which local vertex must send or receive remote color information.
`E/W/N/S color pairs`	Checkerboard data colors for one-hop traffic in each direction.
`relay direction queues`	Circular software relay queues. Incoming wavelets that are not for this PE are buffered here before being forwarded.
`expected_recv`	Host-computed data and done counts used for completion detection.
`done` sentinel	Special wavelet saying a source PE has finished sending this round.
`sync_reduce_c0/c1`, `sync_bcast`	1D barrier colors. In 2D, SW-relay uses in-band control because the N/S data queues occupy the queue budget.

3.4 Route Types



(a) E/W checkerboard

(b) N/S checkerboard

(c) Software multi-hop relay

Figure 3: SW-relay route types.

3.5 Why We Built It This Way

`sw-relay` was the first design that made the whole CSR coloring loop run autonomously on the device. It solved three early problems:

- It removed host mediation from every coloring round.
- It provided a clear per-direction route for every boundary edge.
- It gave us a stable correctness baseline for later kernels.

The cost is that software relay serializes the fabric path. A wavelet crossing ten hops wakes roughly ten relay handlers. The current design was created to remove that per-hop software cost.

4 Design B: Current 2D Segmented LWW Design

4.1 Purpose

`2d_seg2` is the current high-performance design. It uses the pipelined-LWW family with a 2D segmented route layout. It is designed for large 2D grids while staying inside the WSE-3 queue and color budgets.

- Source files: `csl/layout_lww_2d_seg2.csl`, `csl/pe_program_lww_2d_seg2.csl`.
- Runner options: `-routing pipelined-lww -lww-layout 2d_seg2`.
- Default segmentation: `S_row=2`, `S_col=1`.
- Partitioning: row-major monotone block partition.
- Main strength: fabric-pipelined data movement with bounded route resources.
- Main remaining costs: structural SW detours, done-token traffic, and dense barriers.

4.2 Core Idea: Make the Loser Receive the Winner

The host gives each PE a contiguous range of global vertex IDs in row-major PE order. The coloring rule is lower-GID wins. Therefore, for most cross-PE edges, the PE with the larger row-major position is the only side that must receive the other side's color. This removes the need for full bidirectional broadcast.

The remaining hard case is anti-diagonal: the winner can be north-east of the loser. East and south alone do not reach south-west. The current design solves that with:

- east data movement along rows;
- east-to-south relay at nonzero columns;
- a westbound back-channel on every row below row 0.

4.3 16 by 16 Layout Overview

The 16 by 16 figure below shows the roles in the current layout. Row segments have size 2. Odd columns except the final column are row bridges. The column axis uses `S_col=1`, so each non-final row is a one-PE column segment and forwards south on alternating column bridge colors. The last column of rows 1–15 is the back-channel source; column 0 receives the back-channel sink.

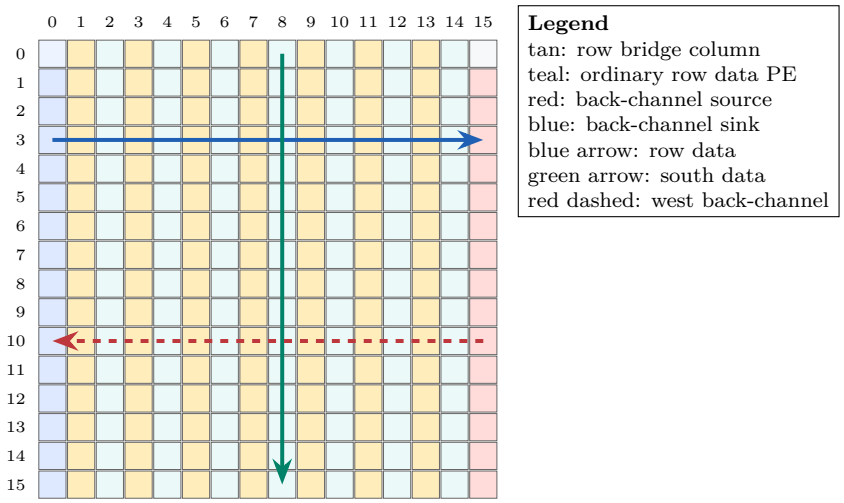
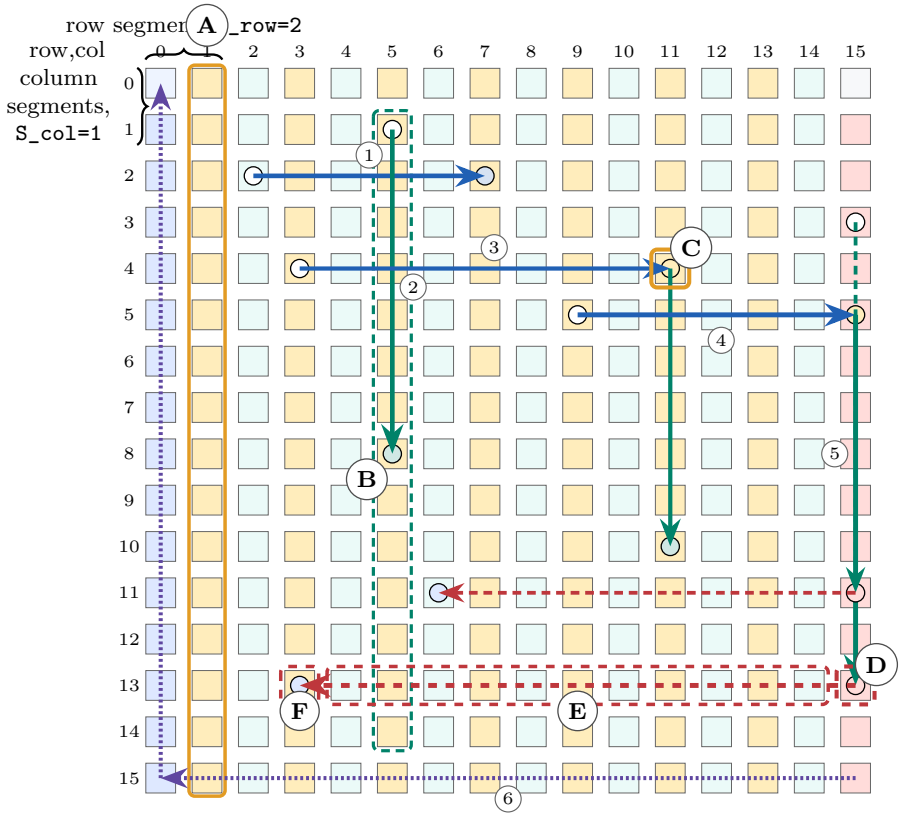


Figure 4: Current `2d_seg2` layout on a 16 by 16 grid. The figure shows roles, not graph vertices. Every PE still stores its own CSR shard.



Role markers on the map			Route line styles	
A		Row bridge columns: re-emit row traffic on `c_be/c_bo`.		East row data on `my_east_color`.
B		Column bridge chain: with `S_col=1`, each row re-emits south traffic.		South data on `my_south_color`.
C		E2S relay: copies an east arrival onto the south stream.		West back-channel on `c_W_back_re/ro`.
D		Back relay: last-column PE injects onto the west back-channel.		Barrier reduce path; broadcasts return in-band on data colors.
E		Back receivers: receive locally while hardware forwards the wavelet west.		
F		Back sink: column-0 side of the back-channel.		
SW		`sw-relay` software relay: every intermediate PE receives and re-sends.		

Numbered routes: 1 same-row east; 2 same-column south; 3 south-east via e2s relay; 4 anti-diagonal SW via east detour, south, and back-channel; 5 SW from last column via direct south into the back relay; 6 row/column barrier. The `sw-relay` sample in the role key shows the baseline software relay: each intermediate PE receives and re-sends.

Figure 5: One-page route map for the current `2d_seg2` 16 by 16 layout. The numbered paths cover the data cases and the barrier; highlighted PEs show where each named component is active.

4.4 Concrete Graph Wavelet Trace

This trace uses a concrete CSR graph with 64 vertices on an 8 by 8 `2d_seg2` fabric. With block partitioning, vertex `gid` maps to `PE gid`, so the selected graph edges are spread across the fabric. The run command was:

```
python3 tools/generate_2d_seg2_wavelet_trace.py \
  --run-id wavelet_trace_2d_seg2_actual_8x8
```

The generator calls the runner's block partitioner with `sw_via_east_backchannel=True` and then validates the same ungated transport model used by `tools/selective_relay_preflight.py`. For this graph the model reaches all 5 required lower-GID-winner deliveries.

Label	CSR edge	Source PE	Consumer PE	Source dir	Case
A	(8, 12)	PE8=(1,0)	PE12=(1,4)	east	same-row east
B	(2, 50)	PE2=(0,2)	PE50=(6,2)	south	same-column south
C	(16, 45)	PE16=(2,0)	PE45=(5,5)	east	south-east via e2s
D	(21, 49)	PE21=(2,5)	PE49=(6,1)	east	anti-diagonal SW, non-last-column source
E	(7, 34)	PE7=(0,7)	PE34=(4,2)	south	anti-diagonal SW, last-column source

Table 3: Actual CSR graph used for the wavelet trace. The source is the lower-GID endpoint, so it is the winner under the LWW rule and its wavelet must reach the higher-GID consumer.

How to read the cycle panels. Each panel is one offline simulator cycle. The uppercase labels A, B, C, D, and E are the five tracked wavelets. Each uppercase marker shows the current position of that wavelet; the matching lowercase marker a, b, c, d, or e is the consumer PE for that wavelet. A star means a receive task has the wavelet and must spend a later step re-emitting it on a row-bridge, column-bridge, e2s, or back-channel color. The cycle numbers are simulator steps for explanation, not WSE-3 performance-counter cycles.

Cycle-by-cycle explanation.

Cycle	What happens in this simulator step
0	A starts at PE8=(1,0); B starts at PE2=(0,2); C starts at PE16=(2,0); D starts at PE21=(2,5); E starts at PE7=(0,7)
1	A moves east on the row stream from PE8=(1,0) to PE9=(1,1); B moves south on the column stream from PE2=(0,2) to PE10=(1,2); C moves east on the row stream from PE16=(2,0) to PE17=(2,1); D moves east on the row stream from PE21=(2,5) to PE23=(2,7); E moves south on the column stream from PE7=(0,7) to PE15=(1,7)
2	A waits at row bridge PE9=(1,1); the receive task re-emits it on the next row segment; B waits at column bridge PE10=(1,2); with S_col=1 each row boundary is a software handoff; C waits at row bridge PE17=(2,1); the receive task re-emits it on the next row segment; D waits at e2s relay PE23=(2,7); the east arrival is re-emitted onto the south stream; E waits at column bridge PE15=(1,7); with S_col=1 each row boundary is a software handoff
3	A moves east on the row stream from PE9=(1,1) to PE11=(1,3); B moves south on the column stream from PE10=(1,2) to PE18=(2,2); C moves east on the row stream from PE17=(2,1) to PE19=(2,3); D moves south on the column stream from PE23=(2,7) to PE31=(3,7); E moves south on the column stream from PE15=(1,7) to PE23=(2,7)

Cycle	What happens in this simulator step
4	A waits at row bridge PE11=(1,3); the receive task re-emits it on the next row segment; B waits at column bridge PE18=(2,2); with S_col=1 each row boundary is a software handoff; C waits at row bridge PE19=(2,3); the receive task re-emits it on the next row segment; D waits at column bridge PE31=(3,7); with S_col=1 each row boundary is a software handoff; E waits at column bridge PE23=(2,7); with S_col=1 each row boundary is a software handoff
5	A reaches consumer PE12=(1,4); B moves south on the column stream from PE18=(2,2) to PE26=(3,2); C moves east on the row stream from PE19=(2,3) to PE21=(2,5); D moves south on the column stream from PE31=(3,7) to PE39=(4,7); E moves south on the column stream from PE23=(2,7) to PE31=(3,7)
6	A is already delivered; B waits at column bridge PE26=(3,2); with S_col=1 each row boundary is a software handoff; C waits at e2s relay PE21=(2,5); the east arrival is re-emitted onto the south stream; D waits at column bridge PE39=(4,7); with S_col=1 each row boundary is a software handoff; E waits at column bridge PE31=(3,7); with S_col=1 each row boundary is a software handoff
7	A is already delivered; B moves south on the column stream from PE26=(3,2) to PE34=(4,2); C moves south on the column stream from PE21=(2,5) to PE29=(3,5); D moves south on the column stream from PE39=(4,7) to PE47=(5,7); E moves south on the column stream from PE31=(3,7) to PE39=(4,7)
8	A is already delivered; B waits at column bridge PE34=(4,2); with S_col=1 each row boundary is a software handoff; C waits at column bridge PE29=(3,5); with S_col=1 each row boundary is a software handoff; D waits at column bridge PE47=(5,7); with S_col=1 each row boundary is a software handoff; E waits at back relay PE39=(4,7); the last-column PE injects the west back-channel next
9	A is already delivered; B moves south on the column stream from PE34=(4,2) to PE42=(5,2); C moves south on the column stream from PE29=(3,5) to PE37=(4,5); D moves south on the column stream from PE47=(5,7) to PE55=(6,7); E reaches consumer PE34=(4,2)
10	A is already delivered; B waits at column bridge PE42=(5,2); with S_col=1 each row boundary is a software handoff; C waits at column bridge PE37=(4,5); with S_col=1 each row boundary is a software handoff; D waits at back relay PE55=(6,7); the last-column PE injects the west back-channel next; E is already delivered
11	A remains delivered at PE12=(1,4); B reaches consumer PE50=(6,2); C reaches consumer PE45=(5,5); D reaches consumer PE49=(6,1); E remains delivered at PE34=(4,2)

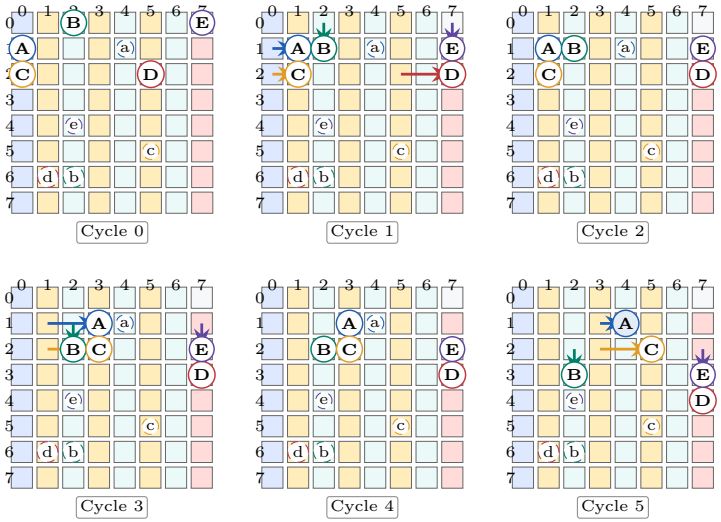


Figure 6: Concrete 2d_seg2 wavelet trace, cycles 0–5.

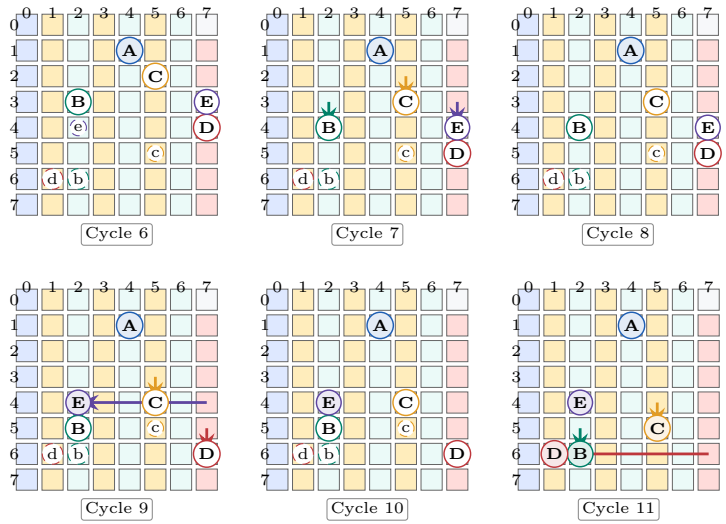


Figure 7: Concrete 2d_seg2 wavelet trace, cycles 6–11.

4.5 Named Components

Component	Role
`c_0`–`c_3`	Row source colors. With the current `S_row=2` default, `c_0` and `c_1` are the normal live source colors; the four-color family keeps the layout generic for smaller single-axis experiments.
`c_be`, `c_bo`	Row bridge colors. Bridges alternate between them so a bridge color is dead before it is reused.
`c_col_0`–`c_col_3`	Column source colors. With the current `S_col=1` default, only the first slot is normally needed; the others are reserved by the generic segmented layout.
`c_col_be`, `c_col_bo`	Column bridge colors. With `S_col=1`, vertical movement is a chain of single-PE bridge segments.
`c_W_back_re`, `c_W_back_ro`	Dedicated westbound back-channel colors, alternated by row parity.
`sync_reduce_c0/c1`	Row reduce colors. They carry the row part of the global "any uncolored?" check.
`sync_col_reduce_c0/c1`	Column reduce colors. They combine row results across rows.
In-band row broadcast	Row broadcast bit rides on the row data stream using an opcode bit.
In-band column broadcast	Column broadcast bit rides on the south data stream using an opcode bit.
`current_level_epoch`	One-bit level tag used to reject stale wavelets left in the fabric across launches.
`boundary_lower_bound()`	Binary-search receive helper over boundary entries sorted by `neighbor_gid`.

4.6 Color Allocation, Queue Budget, and Bridge Roles

The word "color" appears in two different places. A Picasso graph color is the value assigned to a vertex. A WSE fabric color is a hardware route identifier used to bind a route to an input or output queue. The route colors below do not consume graph-color choices; the graph color is packed inside the wavelet payload.

Fabric color IDs	Names	Use
0–3	`c_0`–`c_3`	Row source colors. The same IDs are reused in every row segment because the segment routes do not overlap live spans. Current `S_row=2` normally uses the first two.
4–5	`c_be`, `c_bo`	Row bridge colors. Bridge columns alternate even/odd bridge colors so a bridge color is terminated before it is reused farther east.

Fabric color IDs	Names	Use
6–7	`c_col_be`, `c_col_bo`	Column bridge colors. With `S_col=1`, each non-final row re-emits south traffic on one of these alternating colors.
8–9	`sync_reduce_c0/c1`	Row reduce colors for the per-round "any uncolored?" barrier.
10	`sync_bcast`	Kept for single-axis compatibility. In the current dual-axis layout, row broadcast rides in-band on `my_east_color`.
11–14	`c_col_0`–`c_col_3`	Column source-color family. Under `S_col=1`, this family is mostly dormant, but it remains part of the generic layout.
15, 17	`c_W_back_re`, `c_W_back_ro`	Dedicated westbound back-channel colors. Row parity chooses which one is live, allowing reuse across rows.
18–19	`sync_col_reduce_c0/c1`	Column reduce colors for combining the row-barrier result across rows.
16, 20	free / reserved	No current live binding. Column broadcast was moved in-band on `my_south_color`, so no dedicated column-broadcast color is needed.

WSE-3 gives this kernel six usable queue IDs after the memcpy-reserved queues. The design therefore treats Q2–Q7 as the live queue budget. Input queues and output queues are separate namespaces, but each live queue binding must still obey the WSE rule: one queue is bound to one fabric color.

Queue	Output-side use	Input-side use
Q2	`tx_east_oq`: row data, row bridge reinjection, and in-band row broadcast on `my_east_color`.	`reduce_recv_iq`: row reduce receive.
Q3	`tx_reduce_oq`: row reduce send on `sync_reduce_*`.	`back_recv_iq`: dedicated west back-channel receive on `c_W_back_*`.
Q4	`tx_back_oq`: only at last-column `back_relay` PEs, injects the west back-channel.	`rx_iq_0`: first row-axis receive slot.
Q5	`tx_south_oq`: south data, e2s relay output, column bridge reinjection, and in-band column broadcast on `my_south_color`.	Usually `rx_iq_1` or `col_reduce_recv_iq`, depending on the PE. The static validator checks the exact case.
Q6	`tx_col_reduce_oq`: column reduce send.	Usually `col_reduce_recv_iq` when Q5 is already needed by row data.
Q7	unused by the current output path.	`south_rx_iq_0`: first column-axis receive slot. In larger row-segment settings it can also be a higher row receive slot.

The worst useful case is already tight. On an interior PE with row data, back-channel reception, south reception, row reduce, and column reduce all live, the input side can bind all six queues: Q2 row reduce, Q3 back-channel, Q4/Q5 row data slots, Q6 column reduce, and Q7 south data. This is why the default split is `S_row=2`, `S_col=1`. Moving to `S_col=2` would create another south receive slot and currently collides with Q3 unless another route is redesigned first.

Role	Where it lives	What it does
`sw-relay` software relay	Baseline route PEs along a multi-hop path.	A PE receives a wavelet, runs a task, and re-sends it toward the next hop. It is very general and reliable, but every hop costs software work.
Row bridge	Current `2d_seg2` row bridge columns, for `S_row=2`: columns 1, 3, 5, 7, 9, 11, and 13 on a 16-column row.	Receives the previous row segment's stream and re-emits it on `c_be` or `c_bo`. This lets the row width grow without allocating a new fabric color for every source PE.
Column bridge	Current `2d_seg2` non-final rows when `S_col=1`.	Re-emits south traffic on alternating column bridge colors. The choice `S_col=1` intentionally keeps the south receive slot count to one so Q3 stays free for the back-channel.
E2S relay	Any nonzero-column PE that receives an east data wavelet and is not on the south edge.	Copies selected east-arriving data onto the south stream. This is the turn that lets a source reach south-east consumers and the first leg of the anti-diagonal SW detour.
Back relay	Last column of rows 1–15 in a 16 by 16 grid.	Injects a south-arriving wavelet onto the dedicated west back-channel using `tx_back_oq` and `c_W_back_*`.
Back receiver / sink	Interior columns and column 0 on rows 1–15.	Receives the back-channel wavelet locally. Interior PEs also let the hardware route auto-forward west; they do not software-reinject every hop.

4.7 Route 1: Row Segments

Rows are split into two-column segments. In each segment, the first PE can source row data; the second PE is a bridge unless this is the final segment. The bridge consumes the segment colors and re-emits on an alternating bridge color.

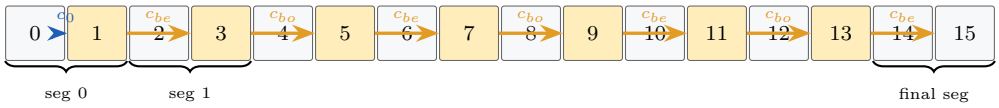


Figure 8: A single row of the 16-column `2d_seg2` grid. Row source and bridge colors are reused; the color count does not grow with the row width.

4.8 Route 2: Column Movement

The column path uses `S_col=1`. This makes the column stream a single-PE segment chain. That choice frees queue Q3 for the dedicated back-channel, which is why the dense 8 by 8 design became stable on hardware.

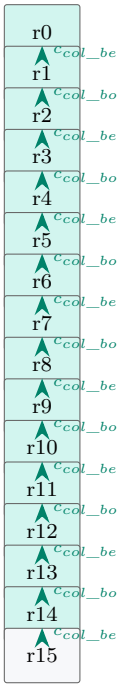


Figure 9: One column in the current default layout. `S_col=1` keeps south receive slots small and avoids the Q3 conflict with the back-channel.

4.9 Route 3: E2S Relay and Back-Channel

The east-to-south relay and westbound back-channel solve the 2D anti-diagonal case. A north-east winner can reach a south-west loser by moving east to the last-column path, south, and then west on the back-channel. The runner should use the east detour only when preflight shows that the fast direct-south path is not enough.

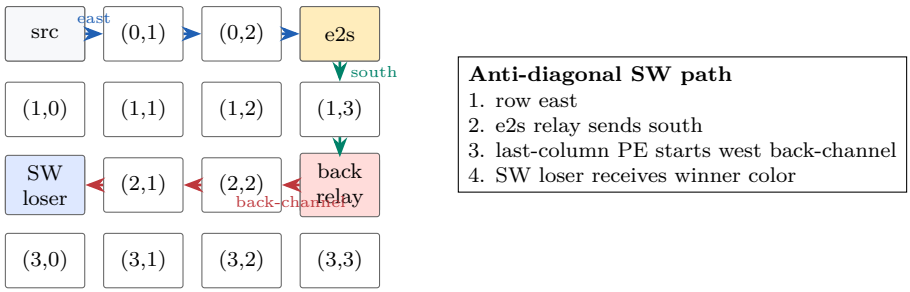


Figure 10: The route used when the winning vertex is north-east of the losing vertex.

4.10 Route 4: Barriers and Broadcasts

`2d_seg2` still follows a BSP round structure. It needs to know when a round is complete and whether another round is necessary. The barrier is split by axis:

- row reduce moves west on `sync_reduce_c0/c1`;
- row broadcast rides in-band on the row data color;
- column reduce moves north on `sync_col_reduce_c0/c1`;
- column broadcast rides in-band on the south data color.

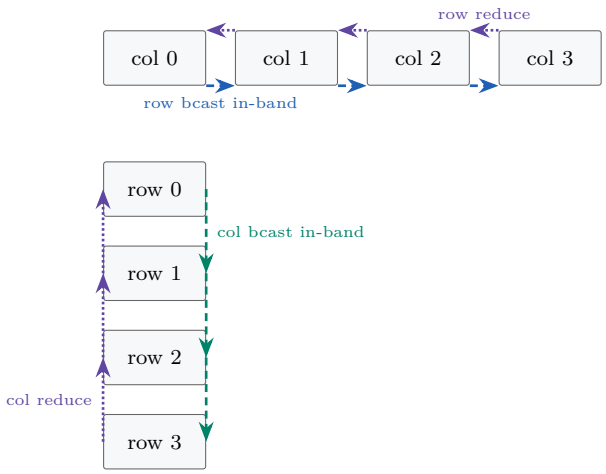


Figure 11: Barrier structure. Reductions use dedicated alternating colors. Broadcasts are packed into existing data streams using opcode bits.

4.11 Wavelet Format

`2d_seg2` uses a 32-bit wavelet with a small type field. The level epoch bit helps reject stale wavelets left in fabric queues after the host starts the next Picasso level.

Wavelet kind	Bits	Meaning
Data	`[31]=0, [30]=parity, [29:8]=gid, [7]=epoch, [6:0]=color`	boundary color update
Data-done	`[31]=1, [30]=parity, [29]=0, [28]=0, [7]=epoch`	source finished this round
Broadcast	`[31]=1, [29]=1, [7]=epoch, [0]=value`	row or column broadcast bit
Reduce	`[31]=1, [28]=1, [7]=epoch, [0]=value`	row reduce bit

4.12 Why We Arrived at This Design

The design is a sequence of constraints becoming visible:

1. `sw-relay` worked but paid software cost at every hop.
2. The first pipelined LWW design used one source color per PE and quickly hit queue and color limits.
3. The monotone block partition removed most westbound data because lower-GID vertices always win.
4. Segments and bridges allowed color reuse. This made 1D wide rows possible.
5. 2D introduced the anti-diagonal SW case. That required the e2s relay and westbound back-channel.
6. Dense 8 by 8 graphs showed that sharing the row reduce path with back-channel data was too congested. The current design restored a dedicated back-channel route.

7. Queue conflicts forced `S_col=1`; this keeps Q3 free for the back-channel.
8. Dense receive-side work became dominant, so boundary arrays were sorted by `neighbor_gid` and searched with binary search.
9. The newest preflight work showed that anti-diagonal SW from a non-last-column source must sometimes route east first. The right policy is automatic: use the fast direct-south path when reachable, and use the east detour only when required.

5 Design Comparison

Question	`sw-relay`	Current `2d_seg2`
How does data move?	One hop at a time. Intermediate PEs receive and forward in software.	Fabric routes carry wavelets through row/column segments; bridges re-emit only at segment boundaries.
Does it need host mediation during rounds?	No. The host launches the kernel and reads results after completion.	No. The host currently launches per Picasso level, while rounds run on device. Removing this per-level launch is the on-device recursion work.
Why is it correct?	It can route in every direction and uses explicit destination information.	Monotone partitioning plus lower-GID-wins reduces required data directions; e2s and back-channel cover the remaining 2D anti-diagonal case.
Why is it slower?	Every multi-hop wavelet wakes several PE tasks.	It still pays for bridge CPU re-emission, done tokens, barriers, and some structural detours.
Best use today	Correctness baseline and comparison target.	Current performance path for 2D hardware scaling.

6 Current Hardware Snapshot

The latest documented hardware matrix after the receive-lookup fix shows that `2d_seg2` is the fastest of the tested paths on the representative rows. Exact values continue to move as the SW routing policy and later performance work change.

Test	Grid	`2d_seg2` cycles	`sw-relay` cycles	Speedup
`test12_many_nodes_20nodes`	4 by 4	189k–197k	707k–714k	about 3.6–3.7x
`test14_random_200nodes`	8 by 8	10.6M	139M	about 13x
`H2_631g_89nodes`	8 by 8	1.99M	14.5M	about 7.3x

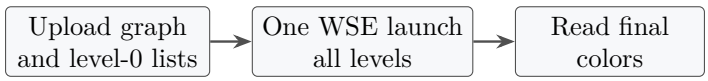
7 Next Performance and System Work

Selective relay was evaluated and should not be the next shipped optimization. The data-only savings were near zero on the measured graphs, and the done stream would still remain ungated.

7.1 On-Device Recursion

On-device recursion should be treated as a main next-work item. Today the round loop is already on the WSE, but the Picasso level loop is still on the host. After each level, the host reads colors back, finds vertices that need another level, creates new color lists, uploads those lists, updates the palette size, and launches the kernel again. On CS-3 hardware, that means one host round trip per level.

The target design is a single device launch for the whole graph:



The required kernel changes are small compared with the routing work:

1. Upload a small `palette_schedule[]` once at the start.
2. Add an on-PE pseudo-random generator to rebuild color lists for vertices that exhausted their list at the current level.
3. Widen the barrier result from one bit to two bits: bit 0 means "some vertex is still uncolored at this level"; bit 1 means "some vertex needs the next level."
4. Add a level-advance task that regenerates lists, resets per-level state, increments `current_level`, and re-enters the same BSP round loop.
5. Keep a `-host-recursion` fallback for debugging and exact comparison with the current path.

This work pairs well with `2d_seg2`. It reduces launch overhead and also removes the cross-launch fabric-residual problem from the normal path, because levels no longer require a fresh host-side kernel launch. We should still keep epoch tagging as a safety feature, but the common case becomes simpler: data stays on the wafer until the full coloring is done.

7.2 Route and Kernel Performance Work

The next route and kernel work should be:

1. **Auto SW routing policy.** Use the direct-south SW path when preflight proves it is reachable; use the east-to-last-column detour only when required.
2. **Done-token aggregation.** Reduce sentinel traffic by aggregating done tokens at segment and axis boundaries.
3. **Weighted monotone partitioning.** Keep the monotone PE order but balance contiguous chunks by edge or boundary weight, not just vertex count.
4. **Palette/list tuning.** Sweep `palette_frac` and `alpha` while respecting the 7-bit color limit in `2d_seg2`.
5. **Receive-state coalescing.** Store one remote color per unique `neighbor_gid`, then map boundary entries to that compact slot.
6. **Direct-SW transport spike.** Explore a route that lets direct-south SW traffic enter a westbound path without detouring to the last column.

8 Source Map

File	Why it matters
`csl/layout.csl`	SW-relay route layout and checkerboard colors.
`csl/pe_program.csl`	SW-relay PE kernel, relay queues, done sentinels, and CSR coloring loop.
`csl/layout_lww_2d_seg2.csl`	Current 2D segmented LWW layout, routes, colors, bridge roles, and queue map.
`csl/pe_program_lww_2d_seg2.csl`	Current 2D segmented LWW kernel, wavelet packing, send/receive tasks, e2s relay, back-channel, barriers.
`picasso/run_csl_tests.py`	Test runner, partitioning, CSR upload, palette/list sizing, and hardware/simulator orchestration.
`tools/selective_relay_preflight.py`	Provenance-correct reachability and traffic model for SW route policy and selective-relay evaluation.
`docs/active/2D_SEG2_DESIGN_REVIEW.md`	Current design risks, latest selective-relay verdict, and performance roadmap.
`docs/active/ON_DEVICE_RECURSION.md`	Design note for moving Picasso's level loop onto the WSE.
`WSE3_Scaling_Analysis.tex`	Earlier long-form scaling note used as background for this document.